AI Assisted Coding Assignment - 10

Chilukala Sai Architha(2303A52220) - Batch 37

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):
    result = 1
    for i in range(1, n):
        result = result * i
    return result
```

Instructions:

1. Run the code and test it with factorial(5).

2. Use an AI assistant to:

o Identify the logical bug in the code.

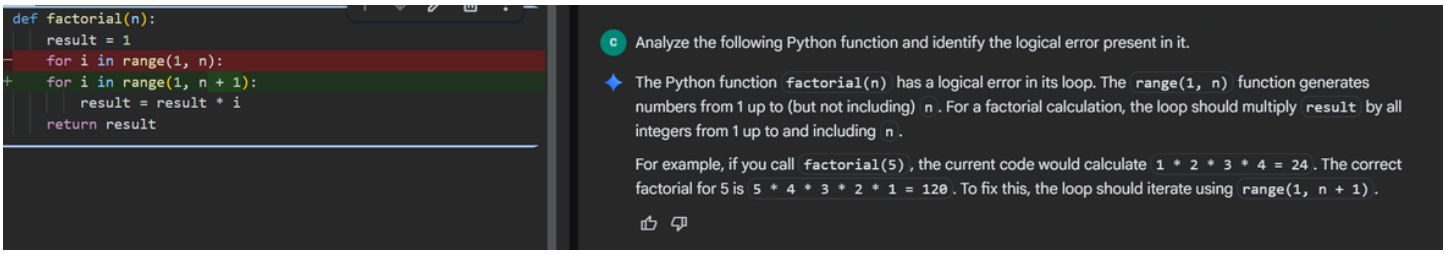
o Explain why the bug occurs (e.g., off-by-one error).

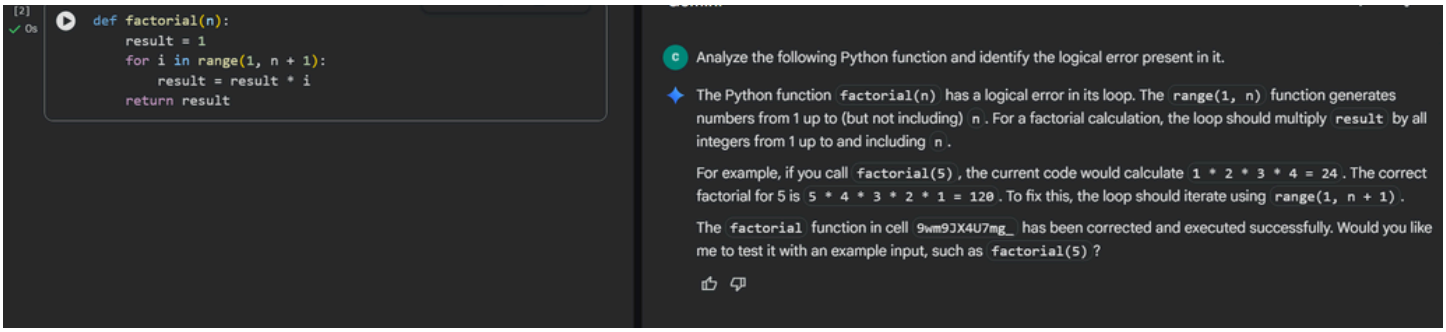
o Provide a corrected version.

3. Compare the AI's corrected code with your own manual fix.

4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output: Corrected function should return 120 for factorial(5)





Prompt Used: Analyze the following Python function and identify the logical error present in it.

Explanation: Through this task, we understood the impact of logical errors, such as off-by-one errors on program correctness. We also learned how AI can assist in identifying bugs and improving the robustness of code by considering edge cases.

Problem Statement 2: Improving Readability & Documentation

Scenario:The following code works but is poorly written:

def calc(a, b, c):

if c == "add":

return a + b

elif c == "sub":

return a - b

elif c == "mul":

return a * b

elif c == "div":

Instructions:

5. Use AI to:

o Critique the function’s readability, parameter naming, and lack of documentation.

o Rewrite the function with:

1. Descriptive function and parameter names.

2. A complete docstring (description, parameters, return value, examples).

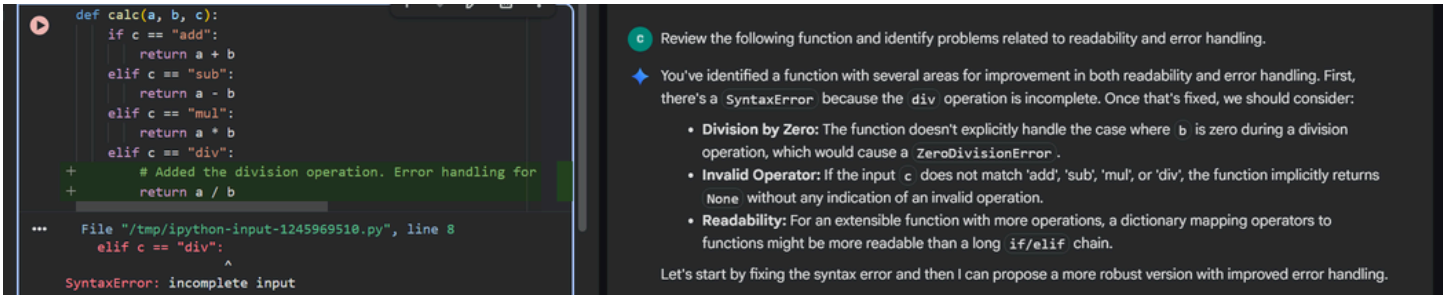
3. Exception handling for division by zero.

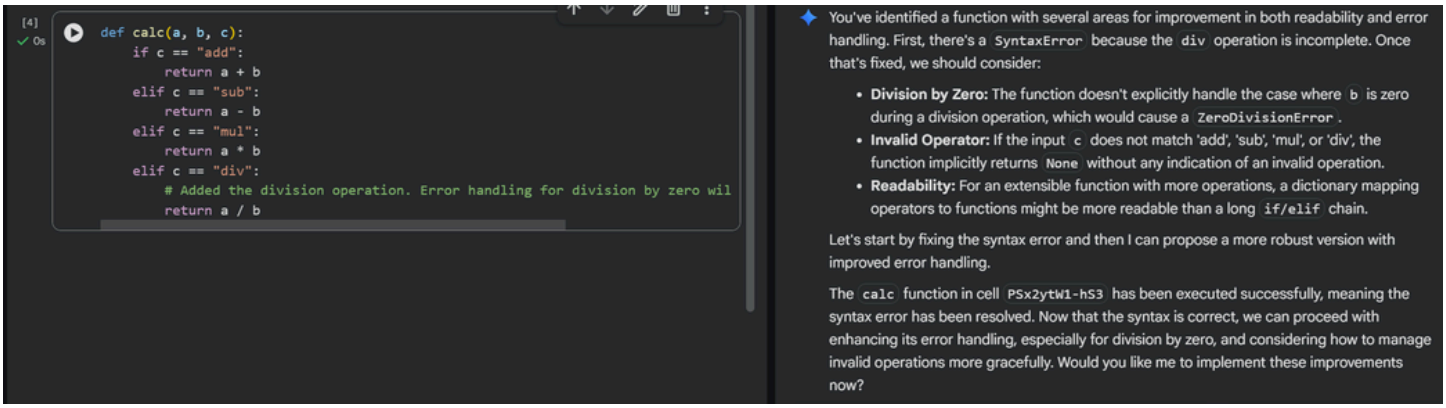
4. Consideration of input validation.

6. Compare the original and AI-improved versions.

7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output: A well-documented, robust, and readable function that handles errors gracefully.





Prompt Used: Review the following function and identify problems related to readability and error handling.

Explanation: This task helped us understand the significance of meaningful naming conventions and proper documentation in enhancing code readability. We learned how AI can support in implementing exception handling and input validation to improve code reliability.

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

def Checkprime(n):

for i in range(2, n):

if n % i == 0:

return False

return True

Instructions:

8. Verify the function works correctly for sample inputs.

9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:

o List all PEP8 violations.

o Refactor the code (function name, spacing, indentation, naming).

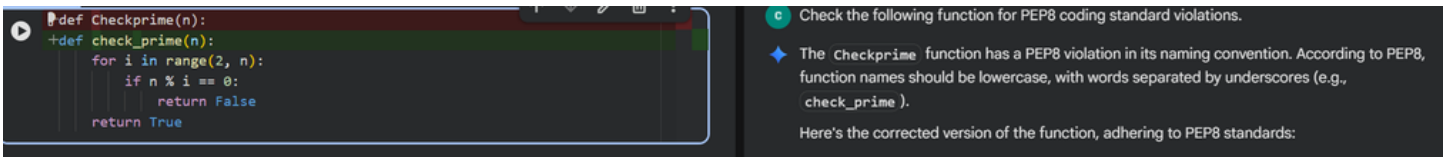
10. Apply the AI-suggested changes and verify functionality is preserved.

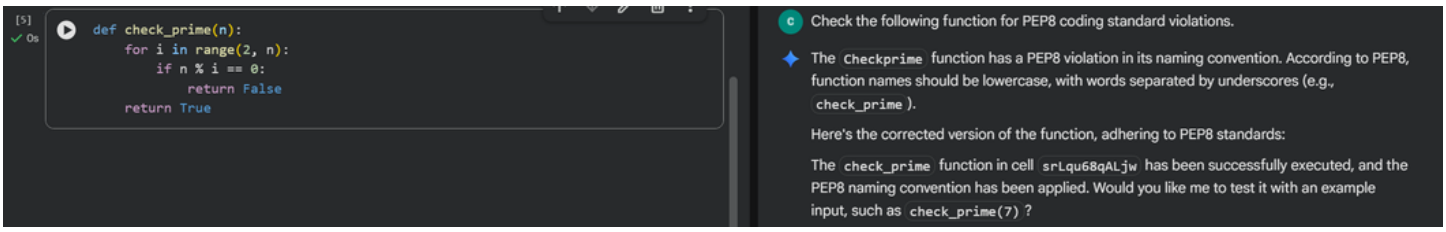
11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def Checkprime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```





Prompt Used: Check the following function for PEP8 coding standard violations.

Explanation: We understood the importance of adhering to PEP8 coding standards for maintaining consistency and readability in Python programs. AI-assisted refactoring enabled us to improve code formatting and naming conventions effectively.

Problem Statement 4: AI as a Code Reviewer in Real Projects

In a GitHub project, a teammate submits:

def processData(d):

return [x * 2 for x in d if x % 2 == 0]

Instructions:

1. Manually review the function for:

o Readability and naming.

o Reusability and modularity.

o Edge cases (non-list input, empty list, non-integer elements).

2. Use AI to generate a code review covering:

a. Better naming and function purpose clarity.

b. Input validation and type hints.

c. Suggestions for generalization (e.g., configurable multiplier).

3. Refactor the function based on AI feedback.

4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent, e.g.:

from typing import List, Union

def double_even_numbers(numbers: List[Union[int,

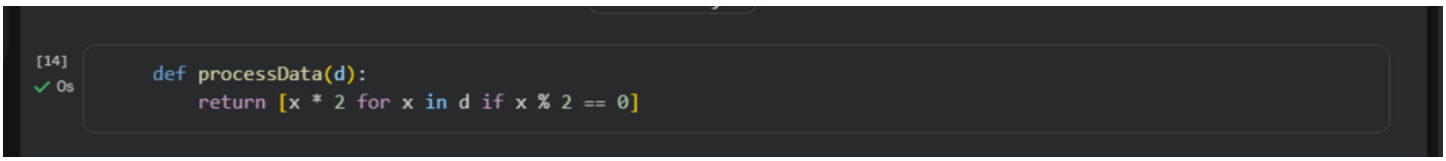
float]]) -> List[Union[int, float]]:

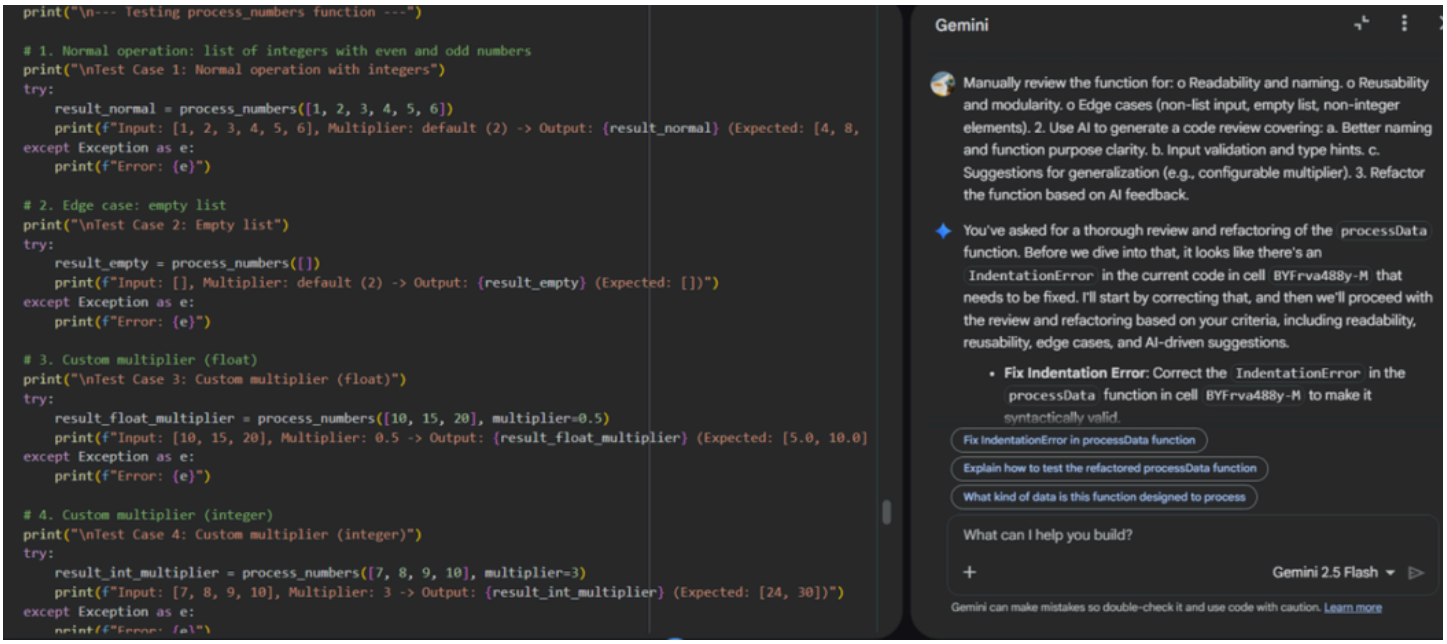
if not isinstance(numbers, list):

raise TypeError("Input must be a list")

return [num * 2 for num in numbers if isinstance(num,

(int, float)) and num % 2 == 0]





```
print("\n--- Testing process_numbers function ---")

# 1. Normal operation: list of integers with even and odd numbers
print("\nTest Case 1: Normal operation with integers")
try:
    result_normal = process_numbers([1, 2, 3, 4, 5, 6])
    print(f"Input: [1, 2, 3, 4, 5, 6], Multiplier: default (2) -> Output: {result_normal} (Expected: [4, 8,
except Exception as e:
    print(f"Error: {e}")

# 2. Edge case: empty list
print("\nTest Case 2: Empty list")
try:
    result_empty = process_numbers([])
    print(f"Input: [], Multiplier: default (2) -> Output: {result_empty} (Expected: [])")
except Exception as e:
    print(f"Error: {e}")

# 3. Custom multiplier (float)
print("\nTest Case 3: Custom multiplier (float)")
try:
    result_float_multiplier = process_numbers([10, 15, 20], multiplier=0.5)
    print(f"Input: [10, 15, 20], Multiplier: 0.5 -> Output: {result_float_multiplier} (Expected: [5.0, 10.0]
except Exception as e:
    print(f"Error: {e}")

# 4. Custom multiplier (integer)
print("\nTest Case 4: Custom multiplier (integer)")
try:
    result_int_multiplier = process_numbers([7, 8, 9, 10], multiplier=3)
    print(f"Input: [7, 8, 9, 10], Multiplier: 3 -> Output: {result_int_multiplier} (Expected: [24, 30])")
except Exception as e:
```

Gemini

Manually review the function for: o Readability and naming. o Reusability and modularity. o Edge cases (non-list input, empty list, non-integer elements). 2. Use AI to generate a code review covering: a. Better naming and function purpose clarity. b. Input validation and type hints. c. Suggestions for generalization (e.g., configurable multiplier). 3. Refactor the function based on AI feedback.

You've asked for a thorough review and refactoring of the `processData` function. Before we dive into that, it looks like there's an `IndentationError` in the current code in cell `BYFrva488y-M` that needs to be fixed. I'll start by correcting that, and then we'll proceed with the review and refactoring based on your criteria, including readability, reusability, edge cases, and AI-driven suggestions.

- **Fix Indentation Error:** Correct the `IndentationError` in the `processData` function in cell `BYFrva488y-M` to make it syntactically valid.

Fix IndentationError in processData function

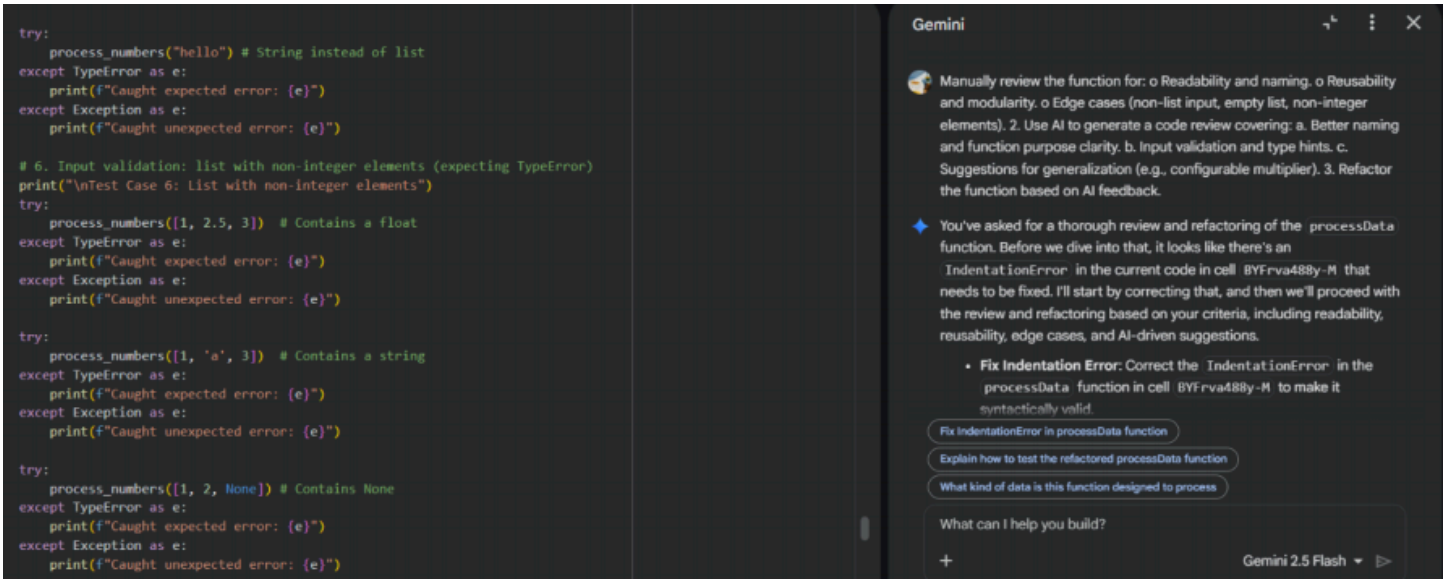
Explain how to test the refactored processData function

What kind of data is this function designed to process

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more



```
try:
    process_numbers("hello") # String instead of list
except TypeError as e:
    print(f"Caught expected error: {e}")
except Exception as e:
    print(f"Caught unexpected error: {e}")

# 6. Input validation: list with non-integer elements (expecting TypeError)
print("\nTest Case 6: List with non-integer elements")
try:
    process_numbers([1, 2.5, 3])  # Contains a float
except TypeError as e:
    print(f"Caught expected error: {e}")
except Exception as e:
    print(f"Caught unexpected error: {e}")

try:
    process_numbers([1, 'a', 3])  # Contains a string
except TypeError as e:
    print(f"Caught expected error: {e}")
except Exception as e:
    print(f"Caught unexpected error: {e}")

try:
    process_numbers([1, 2, None]) # Contains None
except TypeError as e:
    print(f"Caught expected error: {e}")
except Exception as e:
    print(f"Caught unexpected error: {e}")
```

Gemini

Manually review the function for: o Readability and naming. o Reusability and modularity. o Edge cases (non-list input, empty list, non-integer elements). 2. Use AI to generate a code review covering: a. Better naming and function purpose clarity. b. Input validation and type hints. c. Suggestions for generalization (e.g., configurable multiplier). 3. Refactor the function based on AI feedback.

You've asked for a thorough review and refactoring of the `processData` function. Before we dive into that, it looks like there's an `IndentationError` in the current code in cell `BYFrva488y-M` that needs to be fixed. I'll start by correcting that, and then we'll proceed with the review and refactoring based on your criteria, including readability, reusability, edge cases, and AI-driven suggestions.

- **Fix Indentation Error:** Correct the `IndentationError` in the `processData` function in cell `BYFrva488y-M` to make it syntactically valid.

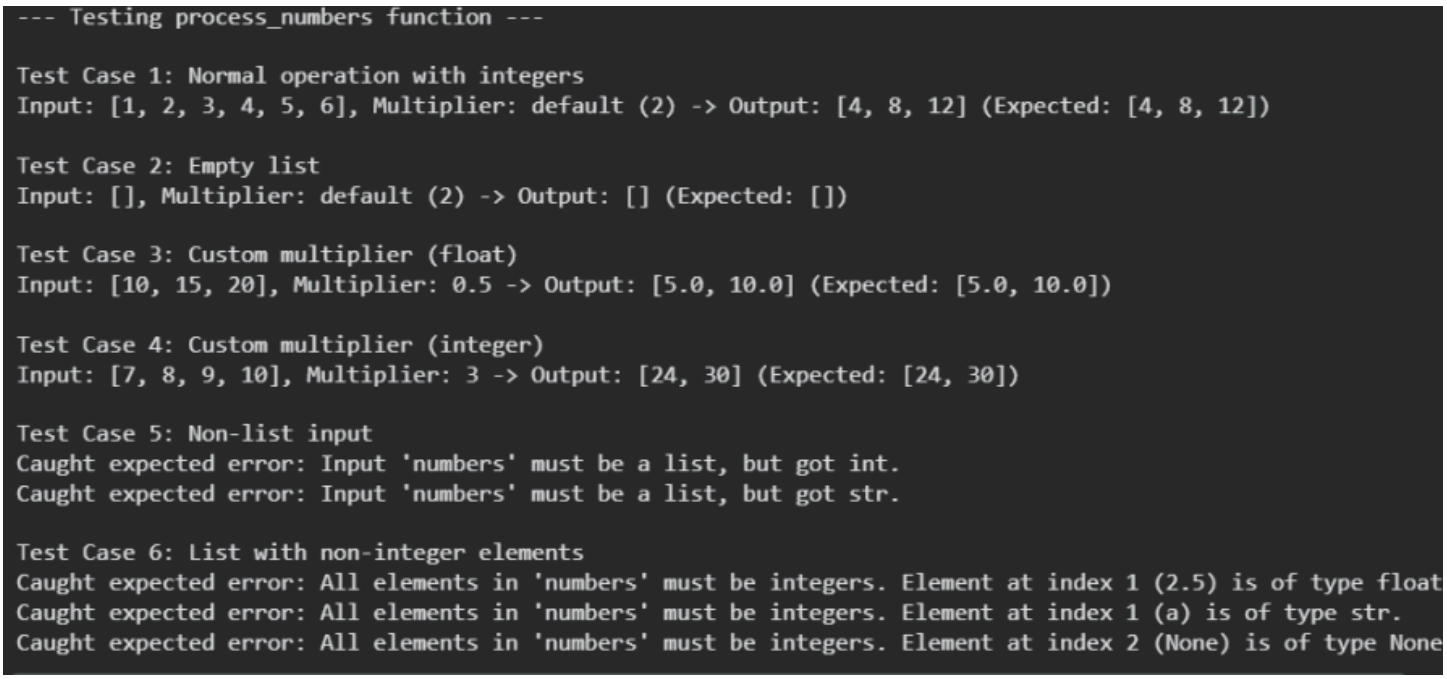
Fix IndentationError in processData function

Explain how to test the refactored processData function

What kind of data is this function designed to process

What can I help you build?

Gemini 2.5 Flash



```
--- Testing process_numbers function ---

Test Case 1: Normal operation with integers
Input: [1, 2, 3, 4, 5, 6], Multiplier: default (2) -> Output: [4, 8, 12] (Expected: [4, 8, 12])

Test Case 2: Empty list
Input: [], Multiplier: default (2) -> Output: [] (Expected: [])

Test Case 3: Custom multiplier (float)
Input: [10, 15, 20], Multiplier: 0.5 -> Output: [5.0, 10.0] (Expected: [5.0, 10.0])

Test Case 4: Custom multiplier (integer)
Input: [7, 8, 9, 10], Multiplier: 3 -> Output: [24, 30] (Expected: [24, 30])

Test Case 5: Non-list input
Caught expected error: Input 'numbers' must be a list, but got int.
Caught expected error: Input 'numbers' must be a list, but got str.

Test Case 6: List with non-integer elements
Caught expected error: All elements in 'numbers' must be integers. Element at index 1 (2.5) is of type float
Caught expected error: All elements in 'numbers' must be integers. Element at index 1 (a) is of type str.
Caught expected error: All elements in 'numbers' must be integers. Element at index 2 (None) is of type None
```

Prompt Used: Manually review the function for: o Readability and naming. o Reusability and modularity. Edge cases (non-list input, empty list, non-integer elements). 2. Use Al to generate

a code review covering: a. Better naming and function purpose clarity. b. Input validation and type hints. c. Suggestions for generalization (e.g., configurable multiplier). 3. Refactor the function based on Al feedback.

Explanation: This task provided insight into improving code reusability, modularity, and input validation in real-world applications. We also learned how AI can assist in enhancing code clarity by adding type hints and improving function structure.

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

def sum_of_squares(numbers):

total = 0

for num in numbers:

total += num ** 2

return total

Instructions:

1. Test the function with a large list (e.g., range(1000000)).

2. Use AI to:

o Analyze time complexity.

o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).

o Provide an optimized version.

3. Compare execution time before and after optimization.

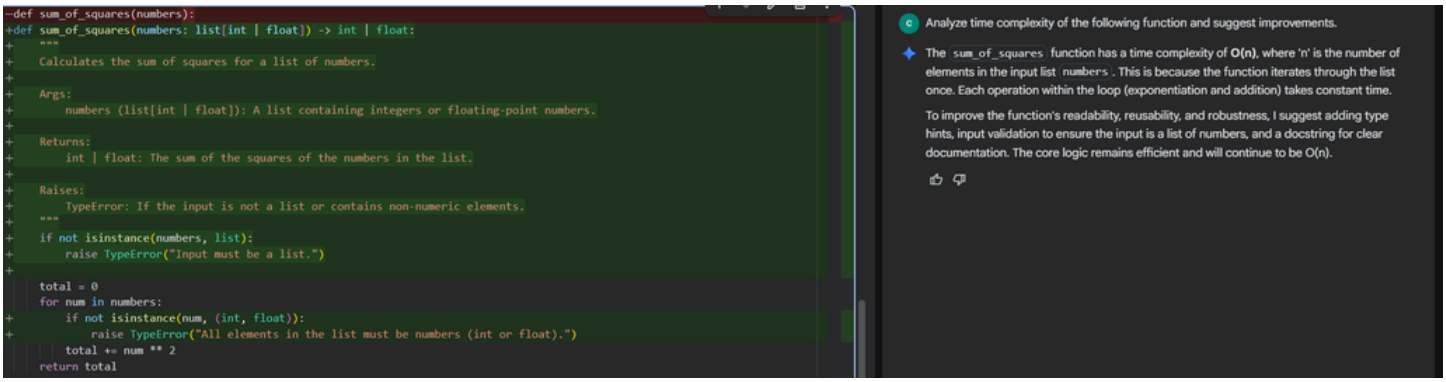
4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

def sum_of_squares_optimized(numbers):

return sum(x * x for x in numbers)



Prompt Used: Analyze the time complexity of the following function and suggest improvements.

Explanation: The role of time complexity in determining program performance for large datasets. AI-based optimization techniques helped in improving execution efficiency while maintaining functionality.